



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Desarrollo de Software Seguro

Integrantes: Mateo Sosa, Marcos Escobar

NRC: 30735

Fecha: May 11, 2026

Contents

Plan Estratégico de Seguridad para SecureFrame Gallery	2
Introducción y Costes de las Vulnerabilidades	2
Coste de una vulnerabilidad de Path Traversal o File Upload Bypass . .	2
Escenario de imagen viral con payload esteganográfico	2
Análisis de Amenazas	2
Amenazas a nivel de Hardware	2
Amenazas a nivel de Código	3
Amenazas a nivel de Diseño	4
Amenazas a nivel de Arquitectura	5
Gestión de Riesgos	6
Matriz de riesgos priorizados	6
Estrategia de tratamiento para riesgos altos	8
Seguridad en el SDLC	8
Fase de Requisitos	8
Fase de Diseño	9
Fase de Desarrollo	9
Fase de Pruebas	9
Fase de Despliegue	10
Alineación con Estándares	10
OWASP ASVS Nivel 2	10
NIST SP 800-218 (SSDF)	10
Justificación Técnica de la Detección Esteganográfica	11
Referencias	11

List of Figures

Plan Estratégico de Seguridad para SecureFrame Gallery

Introducción y Costes de las Vulnerabilidades

SecureFrame Gallery es una galería multimedia pública que almacena imágenes subidas por usuarios. Al combinar recepción de archivos binarios no controlados con visualización masiva, se convierte en un objetivo atractivo para actores maliciosos. El riesgo central no son las vulnerabilidades convencionales (SQL Injection, XSS), sino la **esteganografía**: una técnica que convierte imágenes aparentemente inofensivas en canales encubiertos para exfiltrar datos o distribuir malware sin activar firewalls ni sistemas IDS.

Coste de una vulnerabilidad de Path Traversal o File Upload Bypass

Un **File Upload Bypass** permite subir un ejecutable disfrazado de imagen (p. ej. `shell.php` renombrado como `foto.jpg`) que, al ser servido por el servidor, otorga una shell remota con los privilegios del proceso web. Las consecuencias incluyen compromiso de la base de datos PostgreSQL, exfiltración de credenciales bcrypt, eliminación masiva de archivos en Supabase Storage y daño reputacional. Un **Path Traversal** sobre endpoints de descarga expone el `.env` con credenciales de BD, JWT secret y claves de Supabase, equivalente al compromiso total del sistema. Según IBM Security (2024), el coste medio de una brecha en aplicaciones con manejo de archivos subidos alcanzó los **4,88 millones USD** por incidente en 2024.

Escenario de imagen viral con payload esteganográfico

Un atacante sube un JPEG con datos ocultos en los bits menos significativos (LSB); el sistema lo aprueba y lo distribuye masivamente. Morkel et al. (2005) señalan que la esteganografía *“provides an excellent cover for the transmission of concealed messages”* porque *“the human visual system is unable to detect changes in individual bit values of the pixels”*. La imagen puede contener instrucciones para una botnet, credenciales robadas o fragmentos de malware ensamblables en el receptor, todo distribuido desde una galería pública sin apariencia de amenaza.

Análisis de Amenazas

Las amenazas se clasifican por nivel de arquitectura siguiendo el OWASP Top 10 (OWASP Foundation, 2021b) y MITRE ATT&CK (MITRE Corporation, 2024). Whitman y Mattord (2022) definen las amenazas como *“the categories of potential violations of information security [...] that can affect an organization’s information assets”*.

Amenazas a nivel de Hardware

Amenaza	Descripción	Probabilidad	Impacto	MITRE
DoS por procesamiento intensivo	Subida masiva y concurrente de imágenes cercanas al límite de 10 MB satura CPU/RAM del servidor FastAPI.	Media	Alto	T1499
Agotamiento de almacenamiento temporal	Fallo silencioso en el análisis deja archivos acumulados en uploads_tmp/, agotando inodos del sistema.	Baja	Medio	T1485
Canal lateral en memoria	Imágenes malformadas pueden explotar desbordamientos de búfer en librerías C nativas (libmagic, libjpeg).	Baja	Crítico	T1203

Mitigación: límite 10 MB verificado en memoria antes de escritura; eliminación del temporal en `finally`; rate limiting con SlowAPI.

Amenazas a nivel de Código

Amenaza	Descripción	Probabilidad	Impacto	CWE
Inyección de comandos	Nombre de archivo sin sanitizar en llamadas a subprocess permite inyectar comandos arbitrarios.	Baja	Crítico	CWE-78

Amenaza	Descripción	Probabilidad	Impacto	CWE
XSS en metadatos EXIF	JavaScript embebido en campos EXIF (UserComment, ImageDescription) se ejecuta en visores que no escapan los metadatos.	Media	Alto	CWE-79
Desbordamiento de búfer	Formatos malformados (BMP, TIFF, WEBP) pueden explotar vulnerabilidades no parcheadas en Pillow/OpenCV.	Media	Alto	CWE-120
Path Traversal en nombre de archivo	Nombre sin sanitizar en rutas de disco permite leer archivos fuera del directorio previsto (p. ej. ../../etc/passwd).	Media	Crítico	CWE-22
Exposición de stack trace	FastAPI en modo debug retorna stack traces completos con rutas, versiones y estructura interna.	Media	Medio	CWE-209

Mitigación: UUID como nombre interno (nunca se usa el nombre original en rutas); stripping de EXIF con Pillow; modo debug deshabilitado en producción.

Amenazas a nivel de Diseño

Amenaza	Descripción	Probabilidad	Impacto	OWASP
Escalación de privilegios por IDOR	GET /images/album/{album_id} expone imágenes de álbumes privados si la autorización no verifica el estado del álbum.	Media	Alto	A01:2021
Falta de segregación de roles	Sin separación clara de roles, usuarios sin privilegios pueden ejecutar acciones administrativas.	Baja	Crítico	A01:2021
Race condition en cuarentena	Aprobaciones simultáneas por dos supervisores generan estado inconsistente en la base de datos.	Baja	Medio	A04:2021
Validación solo en frontend	Validación de tipo de archivo únicamente en JavaScript es bypassable con Burp Suite.	Alta	Alto	A05:2021

Mitigación: RBAC con `require_admin` en todos los endpoints; validación MIME en servidor con `python-magic`; queries con `WHERE album_state = 'Aprobado'` para visitantes.

Amenazas a nivel de Arquitectura

Amenaza	Descripción	Probabilidad	Impacto
Archivos en el mismo servidor de la aplicación	Almacenamiento local expone los archivos ante cualquier Path Traversal o LFI sobre FastAPI.	Baja	Crítico

Amenaza	Descripción	Probabilidad	Impacto
Canal esteganográfico no detectado	Falsos negativos permiten que imágenes con datos ocultos se publiquen y distribuyan.	Media	Alto
Compromiso de credenciales de Supabase	Exposición del <code>.env</code> entrega la <code>SUPABASE_KEY</code> con permisos completos sobre BD y storage.	Baja	Crítico
JWT sin revocación	Tokens HS256 son válidos 30 minutos; un token robado es utilizable durante toda su ventana de vida.	Media	Alto

Mitigación: archivos en Supabase Storage (separado del servidor); análisis multicapa LSB + histograma + EOF + EXIF + MIME; `.env` en `.gitignore`; expiración de 30 minutos.

Gestión de Riesgos

El modelo de gestión de riesgos sigue la metodología OWASP Risk Rating (OWASP Foundation, 2023a), estimando el nivel como función de probabilidad de explotación e impacto técnico y de negocio, en línea con los objetivos del NIST SP 800-218 de reducir vulnerabilidades y limitar su impacto (Souppaya et al., 2022).

Matriz de riesgos priorizados

ID	Riesgo	Probabilidad	Impacto	Nivel	Tratamiento
R-01	Falso negativo en detección es-teganográfica	Media	Crítico	Alto	Análisis multicapa LSB + histograma + EOF; cuarentena por defecto; revisión del supervisor

ID	Riesgo	Probabilidad	Impacto	Nivel	Tratamiento
R-02	File Upload Bypass — ejecutable disfrazado de imagen	Baja	Crítico	Alto	Validación MIME por magic numbers; re-encoding con Pillow; UUID como nombre interno
R-03	Escalación de privilegios IDOR	Media	Alto	Alto	RBAC en todos los endpoints; filtros de estado y propietario en queries
R-04	Compromiso de credenciales — exposición del <code>.env</code>	Baja	Crítico	Alto	<code>.env</code> en <code>.gitignore</code> ; rotación de claves; variables de entorno en plataforma
R-05	DoS por procesamiento intensivo	Media	Alto	Medio	Límite 10 MB; rate limiting; timeout en análisis
R-06	XSS almacenado vía metadatos EXIF	Media	Alto	Medio	Stripping de EXIF con Pillow; CSP estricta en cabeceras HTTP
R-07	Sesión hijacking por token JWT robado	Baja	Alto	Medio	Expiración 30 min; HTTPS obligatorio; <code>HttpOnly</code> al migrar a cookies

R-08	Desbordamiento de búfer en librerías de imagen	Baja	Alto	Bajo	Actualización periódica de Pillow/OpenCV; SCA en CI/CD
------	--	------	------	------	--

Estrategia de tratamiento para riesgos altos

R-01 — Falso negativo esteganográfico: ninguna técnica individual es suficiente. Se aplican cuatro análisis complementarios: LSB (desviaciones estadísticas en bits menos significativos), histograma (uniformidad anómala de valores), EOF (datos tras marcadores de fin de archivo) y EXIF (campos de metadatos con riesgo documentado). Un atacante puede evadir el análisis LSB con spreading o transformada DCT (Sarkar & Sanyal, 2014), pero superar las cuatro capas simultáneamente es significativamente más difícil. Las imágenes con cualquier señal sospechosa van a cuarentena para revisión manual del Supervisor.

R-02 — File Upload Bypass: `python-magic` lee los primeros bytes del archivo (magic bytes) en lugar de confiar en la extensión o el `Content-Type` del cliente. Pillow re-codifica la imagen desde cero antes de almacenarla, eliminando contenido anómalo. El nombre en disco es siempre un UUID del servidor, descartando por diseño el path traversal basado en el nombre original.

Seguridad en el SDLC

La seguridad se integró en todas las fases del ciclo de desarrollo siguiendo el NIST SP 800-218 (Souppaya et al., 2022) y el OWASP ASVS Nivel 2 (OWASP Foundation, 2021a). El SSDF estructura las prácticas en cuatro grupos: **Preparar** (PO), **Proteger** (PS), **Producir** (PW) y **Responder** (RV).

Fase de Requisitos

- **Requisitos de seguridad explícitos por RF:** RF03 especificó que la detección de esteganografía requiere análisis estructural del archivo, no solo verificación de extensión o MIME. Este es un requisito de integridad del proceso, no una decisión técnica.
- **Casos de abuso junto a cada caso de uso:** el caso “usuario sube imagen” tiene como contraparte “atacante sube ejecutable disfrazado” y “atacante sube imagen con payload esteganográfico”, alineado con la tarea PO.3.2 del SSDF (Souppaya et al., 2022).
- **Política de contraseñas en RF01:** mínimo 8 caracteres, mayúscula, dígito y carácter especial; almacenamiento con bcrypt y salting, según OWASP ASVS v4 secciones V2 y V3.

Marcos de referencia: OWASP ASVS Nivel 2 (OWASP Foundation, 2021a); NIST SSDF PO.3 (Souppaya et al., 2022).

Fase de Diseño

- **Threat Modeling STRIDE** sobre el endpoint de subida, el flujo de cuarentena y el panel del Supervisor.
- **RBAC desde el diseño:** jerarquía Visitante < Usuario < Supervisor con permisos mínimos definidos antes de escribir código. Visitantes acceden solo a imágenes Limpio de álbumes Aprobado; supervisores tienen acceso privilegiado explícito.
- **Validación multicapa MIME → EXIF → EOF → Histograma → LSB:** cada técnica opera sobre una dimensión distinta del archivo; evadir una no implica evadir las demás.
- **Almacenamiento segregado:** archivos en Supabase Storage (infraestructura separada); un Path Traversal sobre FastAPI no alcanza los archivos de usuarios.

Marcos de referencia: NIST SSDF PW.1 (Souppaya et al., 2022); OWASP Threat Modeling (OWASP Foundation, 2023b).

Fase de Desarrollo

- **Validación con Pydantic:** todos los modelos de entrada (CreateAlbumRequest, RegisterRequest, etc.) validan tipos, longitudes y formatos. `validate_password` aplica regex de complejidad directamente en el modelo.
- **Consultas parametrizadas:** todas las operaciones sobre PostgreSQL usan `psycopg2` con parámetros `%s` posicionales; no existe ninguna query construida por concatenación.
- **Cabeceras de seguridad en middleware:** `add_security_headers` agrega en todas las respuestas `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection: 1; mode=block`, `Content-Security-Policy` y `Referrer-Policy`.
- **Mitigación de timing attacks:** el hash `bcrypt` se evalúa siempre (con hash dummy si el usuario no existe), igualando el tiempo de respuesta para impedir enumeración de cuentas.
- **Archivos temporales en finally:** los temporales de `uploads_tmp/` se eliminan siempre, independientemente del resultado del análisis o de si se lanzó una excepción.

Marcos de referencia: OWASP ASVS V5 y V12 (OWASP Foundation, 2021a); NIST SSDF PW.5 (Souppaya et al., 2022).

Fase de Pruebas

- **Pruebas de bypass MIME:** se subieron archivos PHP, HTML y EXE renombrados con extensión de imagen; el sistema los rechazó en todos los casos.
- **Pruebas de autorización:** endpoints con `require_admin` retornan HTTP 403 ante tokens de rol `Guest`; endpoints públicos no exponen datos restringidos.
- **Revisión de cabeceras:** verificación con herramientas de análisis HTTP de los valores correctos en todas las respuestas.
- **SCA básico:** `requirements.txt` revisado para CVEs conocidos; dependencias actualizadas a versiones estables antes de la entrega.

Recomendación productiva: DAST con OWASP ZAP en staging y SAST con Bandit en CI/CD.

Marcos de referencia: OWASP ASVS V12.3 (OWASP Foundation, 2021a); NIST SSDF PW.7 y PW.8 (Souppaya et al., 2022).

Fase de Despliegue

- **Configuración sensible en variables de entorno:** credenciales gestionadas con `python-dotenv`; `.env` en `.gitignore` y nunca comprometido en el repositorio.
- **CORS restrictivo:** solo orígenes declarados en `ALLOWED_ORIGINS` son aceptados.
- **Política de bucket:** acceso al storage de Supabase exclusivamente vía backend autenticado con `service_role` key; sin acceso directo sin autenticación.

Marcos de referencia: NIST SSDF PO.4 (Souppaya et al., 2022); OWASP ASVS V14 (OWASP Foundation, 2021a).

Alineación con Estándares

OWASP ASVS Nivel 2

El ASVS Nivel 2 está diseñado para “*applications that contain sensitive data, requiring protection*” (OWASP Foundation, 2021a). La siguiente tabla documenta la cobertura de los capítulos más relevantes:

Capítulo ASVS	Requisito	Control implementado
V2 — Autenticación	V2.1: Contraseñas ≥ 8 chars, complejidad	Validador <code>validate_password</code> en Pydantic + <code>bcrypt</code>
V3 — Sesiones	V3.5: Tokens con expiración	JWT HS256 con <code>exp</code> de 30 minutos
V4 — Control de acceso	V4.1: Mínimo privilegio	RBAC con <code>get_current_user</code> / <code>require_admin</code>
V5 — Validación	V5.1: Validación en servidor	Pydantic en todos los modelos de entrada
V9 — Comunicaciones	V9.1: TLS en tránsito	HTTPS via Supabase + cabecera HSTS recomendada
V12 — Archivos	V12.1: Límite de tamaño; V12.2: Validación MIME	10 MB limit + <code>python-magic</code> + re-encoding Pillow
V14 — Configuración	V14.4: Cabeceras de seguridad HTTP	Middleware con CSP, XCTO, XFO, Referrer-Policy

NIST SP 800-218 (SSDF)

Práctica SSDF	Grupo	Implementación en SecureFrame Gallery
PO.3.2 — Requisitos de seguridad documentados	Preparar	RF01-RF05 con controles de seguridad explícitos
PW.1.1 — Threat modeling	Producir	STRIDE aplicado en fase de diseño
PW.5.1 — Validación de entradas	Producir	Pydantic + magic numbers + consultas parametrizadas
PW.7.2 — Pruebas de seguridad	Producir	Pruebas de bypass MIME, IDOR y autorización
PS.1.1 — Protección del repositorio	Proteger	.env en .gitignore; no credenciales en código
RV.1.1 — Gestión de vulnerabilidades	Responder	Revisión de dependencias; flujo de cuarentena

Justificación Técnica de la Detección Esteganográfica

La detección se implementa con técnicas propias sobre `numpy`, `OpenCV` y `Pillow`, sin depender de una librería especializada, por tres razones:

1. **Independencia de formato:** herramientas como `stegdetect` solo reconocen firmas de algoritmos conocidos. Las técnicas propias (LSB ratio, varianza de histograma, EOF) detectan anomalías estadísticas independientemente del método de ocultamiento empleado (Sarkar & Sanyal, 2014).
2. **Control de falsos positivos:** los umbrales (`lsb_ratio > 0.52 or < 0.48`; `variance < 5000`) son ajustables según la distribución real de imágenes del sistema, lo que evita saturar la bandeja del Supervisor. Este enfoque es coherente con los hallazgos de Aljughaiman y Alrawashdeh (2025) sobre el impacto de la distribución de bits ocultos en las métricas estadísticas de detección.
3. **Transparencia:** cada técnica retorna el motivo específico del marcado, almacenado en `image_analysis` (JSONB), permitiendo que el Supervisor tome decisiones informadas en lugar de actuar sobre un veredicto opaco.

La librería `stegano` se usa exclusivamente para generar imágenes de prueba; no forma parte del pipeline de producción.

Referencias

- Aljughaiman, A., & Alrawashdeh, R. (2025). Content-adaptive LSB steganography with saliency fusion, ACO dispersion, and hybrid encryption with ablation study. *Scientific Reports*, 15. <https://doi.org/10.1038/s41598-025-33920-9>
- IBM Security. (2024). *Cost of a Data Breach Report 2024*. IBM Corporation. <https://www.ibm.com/reports/data-breach>
- MITRE Corporation. (2024). *ATT&CK® Enterprise Matrix*. <https://attack.mitre.org/>
- Morkel, T., Eloff, J. H. P., & Olivier, M. S. (2005). An overview of image steganography. *Proceedings of the ISSA 2005 New Knowledge Today Conference*, 1–11.

<https://mo.co.za/open/stegoverview.pdf>

OWASP Foundation. (2021a). *Application Security Verification Standard (ASVS) 4.0.3*. <https://owasp.org/www-project-application-security-verification-standard/>

OWASP Foundation. (2021b). *OWASP Top 10:2021 — The ten most critical web application security risks*. <https://owasp.org/Top10/>

OWASP Foundation. (2023a). *OWASP risk rating methodology*. https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

OWASP Foundation. (2023b). *Threat modeling*. https://owasp.org/www-community/Threat_Modeling

Sarkar, T., & Sanyal, S. (2014). *Steganalysis: Detecting LSB steganographic techniques* [Preprint]. arXiv. <https://arxiv.org/abs/1405.5119>

Souppaya, M., Morello, J., & Scarfone, K. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST SP 800-218). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>

Whitman, M. E., & Mattord, H. J. (2022). *Principles of information security* (7.th ed.). Cengage Learning.